🎓 جزوه طلایی سیستم‌عامل

فصل ۱۰: حافظه مجازی - بخش پنجم (تکنیک‌های پیشرفته و بافرسازی)

۱. الگوریتم‌های مبتنی بر شمارش (Counting-Based Algorithms)

علاوه بر زمان (LRU/FIFO)، می‌توانیم بر اساس **تعداد دفعات استفاده** تصمیم بگیریم. برای هر صفحه یک شمارنده (Counter) نگه می‌داریم.

الف) الگوریتم LFU (Least Frequently Used)

- **منطق:** صفحه‌ای که **کمترین** تعداد مراجعه را داشته، حذف کن.
- **استدلال:** صفحه‌ای که زیاد استفاده شده (عدد شمارنده بالا)، حتماً مهم است و باید بماند.
- **مشکل بزرگ:** ممکن است یک صفحه در ابتدای برنامه خیلی زیاد استفاده شود (شمارنده بالا برود) اما دیگر هرگز نیاز نشود. به خاطر شمارنده بالا، در حافظه می‌ماند و فضا را اشغال می‌کند.
    - *راهکار:* **شیفت دادن (Aging):** در فواصل زمانی، شمارنده‌ها را نصف کنیم تا اثر گذشته کم‌رنگ شود.

ب) الگوریتم MFU (Most Frequently Used)

- **منطق:** صفحه‌ای که **بیشترین** تعداد مراجعه را داشته، حذف کن.
- **استدلال:** صفحه‌ای که شمارنده‌اش کم است، احتمالا **تازه وارد شده** و هنوز فرصت نکرده استفاده شود، پس نباید حذف شود.

⚠ **نکته کنکوری:** هیچ‌کدام از LFU و MFU در عمل رایج نیستند چون پیاده‌سازی گرانی دارند و تقریب خوبی از الگوریتم بهینه (OPT) نیستند.

🚀 ۲. الگوریتم‌های بافرسازی صفحه (Page-Buffering Algorithms)

هدف این تکنیک‌ها، **پوشاندن کندی سرعت دیسک** و افزایش سرعت پاسخ‌دهی سیستم است.

تکنیک ۱: انبار قاب‌های آزاد (Pool of Free Frames)

- وقتی خطای صفحه رخ می‌دهد، منتظر نمی‌مانیم تا قربانی در دیسک نوشته شود (Page Out).
- **روش:** صفحه جدید را سریعاً در یک **قاب آزاد** (از انبار) بارگذاری می‌کنیم و پردازه را اجرا می‌کنیم.
- *سپس در پس‌زمینه:* صفحه قربانی را در دیسک می‌نویسیم و قابش را به انبار آزاد برمی‌گردانیم.

تکنیک ۲: لیست صفحات تغییریافته (Modified Page List)

- وقتی دیسک بیکار است، سیستم‌عامل صفحاتی که تغییر کرده‌اند (Dirty) را در دیسک می‌نویسد و بیت تغییر (Modify Bit) آن‌ها را **صفر** می‌کند.
- **مزیت:** وقتی نیاز به انتخاب قربانی شد، احتمال پیدا کردن صفحه "تمیز" (Clean) بالا می‌رود و نیازی به نوشتن آن لحظه‌ای نیست (چون قبلاً نوشته شده).

تکنیک ۳: بازیافت از قاب آزاد (Reclaim from Free Frame)

- وقتی قابی آزاد می‌شود، محتویات آن را بلافاصله پاک نمی‌کنیم.
- اگر پردازه دوباره همان صفحه را بخواهد و قاب هنوز بازنویسی نشده باشد، آن را از لیست آزاد **پس می‌گیریم** (Reclaim).
- **نتیجه:** بدون هیچ I/O دیسکی، صفحه دوباره زنده می‌شود (مثل کثیف کردن زباله قبل از آمدن ماشین شهرداری!).

۳. برنامه‌های کاربردی و مدیریت حافظه (Applications & Page Replacement)

آیا همیشه سیستم‌عامل بهتر از برنامه می‌داند چه چیزی را نگه دارد؟ **خیر!**

مشکل دیتابیس‌ها (Database & Data Warehouses):

برنامه‌هایی مثل پایگاه‌داده خودشان مدیریت حافظه و بافرینگ دارند.

- **مشکل بافرینگ دوبل (Double Buffering):** هم دیتابیس دیتا را در بافر خودش نگه می‌دارد، هم سیستم‌عامل در کش خودش. (اتلاف حافظه).
- **مشکل الگوریتم:** مثلاً دیتابیس می‌داند که دارد یک جدول را خطی می‌خواند (Sequential Scan) و صفحات قبلی دیگر نیاز نیستند، اما سیستم‌عامل با الگوریتم LRU سعی می‌کند آن‌ها را نگه دارد! در اینجا **MFU** برای دیتابیس بهتر از LRU است، اما سیستم‌عامل نمی‌داند.

راه‌حل: دیسک خام (Raw Disk / Raw I/O)

- سیستم‌عامل به برخی برنامه‌های خاص اجازه می‌دهد دیسک را به صورت یک **آرایه خام از بلاک‌ها** ببینند.
- تمام سرویس‌های فایل‌سیستم (بافرینگ، قفل‌گذاری، پیش‌خوانی و...) **دور زده می‌شود (Bypass)**.
- برنامه خودش مسئولیت مدیریت دقیق حافظه را بر عهده می‌گیرد.

📝 خلاصه نکات طلایی بخش پنجم:

1. **LFU:** کمترین استفاده حذف شود (مشکل: صفحات قدیمی پرمصرف می‌مانند).
2. **MFU:** بیشترین استفاده حذف شود (فرض: صفحات کم‌استفاده، جدید و مهم‌اند).
3. **Pool of Free Frames:** برای سرعت بخشیدن، اول صفحه جدید را می‌آوریم، بعد قربانی را می‌نویسیم.
4. **Raw I/O:** دسترسی مستقیم به دیسک بدون دخالت فایل‌سیستم (مخصوص دیتابیس‌ها برای جلوگیری از سربار اضافی).